Web Front-End

By Jawad Jammoul

Content:

Chapter 01 : Html

01*Definitions:

Markup languages	Used to structure and describe content (like text, images)
Scripting languages	Used to add behavior and automate tasks

HTML	A markup language used to create and structure the content of web pages.

Interpreter	Executes code line by line at runtime, without creating a separate executable file.
Compiler	Translates the entire code into machine language first

Static website	Shows fixed content that stays the same for every user
Dynamic website	Generates content in real time based on user interaction, data, or server logic.

Responsive website	A website that automatically adapts its layout and design t

Domain name	The human-readable address of a website (like google.com).
Hosting	A service that stores website files and makes them accessible
DomainNameSystem	Translates a domain name into an IP address
SearchEngine Optimization	Techniques used to improve a website's visibility in search engine results.

HTML 1.0 (1991) :	The first version, very basic with text and links only.
HTML 2.0 (1995) :	The first official standard, added support for forms.
HTML 3.2 (1997) :	Introduced tables and improved layout features.
HTML 4.01 (1999) :	Separated content from design using CSS.
XHTML (2000) :	A stricter, XML-based version of HTML with cleaner syntax.
HTML5 (2014) :	The modern standard with multimedia , and better web app support.

02*Document Structure:

<!DOCTYPE html> :	Declares the document type and tells the browser to use HTML5 standards.
<html> :	The root element that wraps the entire HTML document.
<head> :	Contains metadata, links, and settings that are not shown on the page.
<body> :	Holds all the visible content displayed in the browser.
<title> :	Sets the page title shown in the browser tab.
<meta> :	Provides metadata like charset, viewport, or description for the page.
<link> :	Connects external resources like CSS files to the document.
<style> :	Defines internal CSS styles for the page.
<script> :	Embeds or links JavaScript code for functionality.
<base> :	Specifies the base URL for all relative links in the document.

```
<!DOCTYPE html>
<html>
<head>
   <title>…</title>
   <meta charset="…">
   <meta name="…" content="width=device-width, initial-scale=1.0">
   <link rel="stylesheet" href="fileName.css">
</head>
<body>
   <h1>…</h1>
   <p>...</p>
   <button onclick="…">…</button>
   <script src="fileName.js"></script>
</body>
</html>
```

03*Text Content & Formatting:

<p>	Defines a paragraph of text.
<pre>	Displays text in preformatted form, preserving spaces and line breaks.
<h1> - <h6>	Define headings, from largest (h1) to smallest (h6).
** **	Inserts a single line break.
<hr>	Creates a horizontal line to separate content.
<b>	Makes text bold without adding importance.
****	Marks text as important, usually displayed in bold.
<i>	Displays text in italics without extra meaning.
****	Emphasizes text, usually shown in italics.
<mark>	Highlights text with a background color.
<small>	Displays smaller-sized text.
<del>	Shows deleted text with a strikethrough.
<ins>	Indicates inserted text, usually underlined.
<sub>	Displays subscript text (below the line).
<sup>	Displays superscript text (above the line).
<q>	Defines a short inline quotation.
<abbr>	Represents an abbreviation or acronym.
<address>	Provides contact information for the author or owner.
<cite>	Refers to the title of a work
<bdo>	Overrides the text direction (e.g., right-to-left).
****	A generic inline container for styling or scripting.
<var>	Represents a variable in programming or math.
<code>	Displays code snippets in a monospace font.
<kbd>	Represents user input (like keyboard input).
<samp>	Shows sample output from a program.
<dfn>	Indicates the defining instance of a term.

```
<body>
  <h2>…</h2>
  <p>…<b>…</b>…<strong>…</strong> </p>
  <p>…<i>…</i>…<em>…</em></p>
  <p>…<mark>…</mark>…</p>
  <p>…<small>…</small>…<p>
  <p>...<del>…</del>…<ins>…</ins> …</p>
  <p>...<sup>…</sup>…<sub>…</sub>…</p>
  </p>
  <hr>
  <p>…<br>
    …
  </p>
  <pre>
    …
    …
  </pre>
</body>
```

```
<body>
  <h2>…</h2>
  <p> <q> … </q> </p>
  <p> … <abbr title =  " … " >…</abbr> … </p>
  <address> …. </address>
  <br>
  <p> … <span style = "…">…</span> </p>
  <p> … <bdo dir = "…">…</bdo> </p>
  <p> … <var>…</var> … </p>
  <p> … <code> … </code> … </p>
  <p>…<samp>…</samp>…</p>
  <p>...<dfn>…</dfn>…</p>
</body>
```

04*Links & Navigation:

<a>	Creates a hyperlink to another page or resource.
<nav>	Defines a section that contains navigation links.
<iframe>	Embeds another webpage or content inside the current page.

```
<body>
  <nav>
    <a href = "…">…</a>
    <a href = "…">…</a>
    <a href = "…">…</a>
  </nav>
  <p>…</p>
  <iframe src = "…" width = "…" height = "…"> </iframe>
```

05*Media (Audio / Video / Embed):

<video>	Embeds video content in the webpage.
<audio>	Embeds sound or music in the webpage.
<source>	Specifies multiple media sources for <video> or <audio>.
<track>	Adds subtitles, captions, or other text tracks to media.
<object>	Embeds external content like PDFs or multimedia.
<embed>	Embeds external content or plugins directly into the page.

```
<body>
 <h2>…</h2>
 <video width = "…" height = "…" controls>
  <source src = "…" type = "…">
  <source src = "…" type = "…">
  <track src = "…" kind= "…" srclang = "…" label = "…">
  <track src = "…" kind= "…" srclang = "…" label = "…">
 </video>
 <h2>…</h2>
  <audio controls>
    <source src = "…" type = "…">
    <source src = "…" type = "…">
  </audio>
  <h2>…</h2>
  <object data = "…" width = "…" height = "…">…</object>
  <h2>…</h2>
  <embed src = "…" width = "…" height = "…">
 </body>
```

06*Graphics (SVG / Canvas):

<canvas>	Provides a drawable area for graphics using JavaScript.
<svg>	Defines scalable vector graphics in XML format.
<circle>	Draws a circle inside an SVG.
<rect>	Draws a rectangle inside an SVG.
<polygon>	Draws a shape with multiple straight sides in SVG.
<defs>	Stores reusable SVG elements that are not directly displayed.
<linearGradient>	Defines a linear color gradient in SVG.
<text>	Displays text inside an SVG graphic.

```
<body>
 <h2>…</h2>
 <canvas id = "…" width = "… "  height= "…"  style="…"></canvas>
<h2>…</h2>
<svg width= "…"  height= "…">
  <defs>
    <linearGradient id = "…" x1 = "…" y1 = "…"  x2 = "…" y2 = "…">
      <stop offset = "…" style = "…" />
      <stop offset = "…" style = "…" />
    </linearGradient>
  </defs>
  <rect x= "…" y= "…" width = "…" height = "80" fill = "…" />
  <circle cx = "…" cy = "…" r = "…" fill = "…" />
  <polygon points = ".. , .. , .. , .." fill = "…" />
  <text x = "…" y = "…" fill = "…">…</text> </svg> </body>
```

07*Images & Mapping:

<picture>	Provides multiple image sources for responsive images
<map>	Defines an image map with clickable areas.
<area>	Specifies a clickable region inside an image map.

```
<body>
  <h2>…</h2>
  <picture>
    <source media = "…" srcset = "…">
    <source media = "…" srcset = "…">
    <img src = "…" alt = "…" width = "…" >
  </picture>
  <h2>…</h2>
  <img src = "…" alt = "…" usemap = "…" width = "…">
  <map name = "…">
      <area shape = "…" coords = "…" href = "…" alt = "…">
      <area shape = "…" coords = "…" href = "…" alt = "…">
  </map>
</body>
```

08*Lists:

** **	Defines an ordered (numbered) list.
** **	Defines an unordered (bulleted) list.
** **	Represents a list item inside or .
<dl>	Defines a description list for terms and their details.
<dt>	Represents a term or name in a description list.
<dd>	Provides the description or definition of a term.
<details>	Creates a collapsible section that can be expanded by the user.
<summary>	Defines the visible heading for a <details> element.

```
<body>
  <h2>…</h2>
  <ol>
    <li>…</li>
    <li>…</li>
  </ol>
  <h2>…</h2>
  <ul>
    <li>…</li>
    <li>…</li>
  </ul>
  <h2>…</h2>
  <dl>
    <dt>...</dt>
    <dd>… </dd>
  </dl>
  <h2>…</h2>
  <details>
    <summary>…</summary>
    <p>…</p>
  </details>
</body>
```

09*Tables:

<table>	Defines a table for organizing data in rows and columns.
<tr>	Defines a table row.
<th>	Defines a header cell in a table, usually bold and centered.
<td>	Defines a standard data cell in a table.
<caption>	Provides a title or description for the table.
<colgroup>	Groups columns to apply styles or properties to them.
<thead>	Contains the header section of a table.
<tbody>	Contains the main body/content of the table.
<tfoot>	Contains the footer section of a table

```
<body>
  <table border = "…">
    <caption>…</caption>
    <colgroup>
      <col style = "…">
      <col style = "…">
    </colgroup>
    <thead>
      <tr>
        <th>…</th>
        <th>…</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>…</td>
        <td>…</td>
      </tr>
      <tr>
        <td>…</td>
        <td>…</td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td>…</td>
        <td>…</td>
    </tfoot>
    </table>
</body>
```

10*Forms:

<form>	Creates a form to collect and send user input.
<input>	Defines an input field for user data (text, password, etc.).
<button>	Creates a clickable button for actions or form submission.
<label>	Defines a label for an input element to improve usability.
<select>	Creates a dropdown list for selecting options.
<option>	Defines an option inside a <select> dropdown.
<optgroup>	Groups related options inside a dropdown list.
<fieldset>	Groups related form elements together visually and semantically.
<legend>	Provides a title for a <fieldset> group.
<datalist>	Provides a list of predefined options for an input field.
<output>	Displays the result of a calculation or user action.

```
<body>
  <form>
    <fieldset>
      <legend>…</legend>
      <label for = "…">…</label>
      <input type = "…" id = "…" list = "…">
      <datalist id = "…">
        <option value = "…">
        <option value = "…">
      </datalist>
      <br>
      <label for = "…">…</label>
      <select id = "…">
        <optgroup label = "…">
          <option>…</option>
          <option>…</option>
        </optgroup>
      </select>
      <output name = "…"></output>
    </fieldset>
  </form>
</body>
```

Input types:

<input type = "Text">	**<input type = "time" >**	**<input type = "file">**
<input type = "password">	**<input type = "datetime-local">**	**<input type = "hidden">**
<input type = "email"	**<input type = "month">**	**<input type = "submit">**
<input type = "number">	**<input type = "week">**	**<input type = "reset">**
<input type = "tel">	**<input type = "color">**	**<input type = "button">**
<input type = "url" >	**<input type = "range" >**	**<input type = "image">**
<input type = "search">	**<input type = "checkbox">**	**<input type = "text" disabled>**
<input type = "date">	**<input type = "radio">**	**<input type = "text" required>**

11*Semantic Layout:

<header>	Represents the introductory content or top section of a page or section.
<footer>	Represents the bottom section of a page or section, usually for links or info.
<main>	Contains the main content of the document, unique to the page.
<section>	Defines a thematic grouping of content within a page.
<article>	Represents a self-contained piece of content like a blog post or news item.
<aside>	Contains side content related to the main content, like ads or notes.
<figure>	Wraps media content such as images, diagrams, or illustrations.
<figcaption>	Provides a caption or description for a <figure> element.

```
<body>
 <header><h1>…</h1></header>
 <main>
  <section>
   <h2>…</h2>
    <article>
     <h3>…</h3>
     <p> … </p>
    </article>
   </section>
  <aside>
   <h3>…</h3>
   <p>…</p>
  </aside>
   <figure>
    <img src = "…" alt = "…" width = "…">
    <figcaption> … </figcaption>
   </figure>
 </main>
 <footer>
  <p>…</p>
 </footer>
</body>
```

12*Attributes:

id	Gives a unique identifier to an element (must be unique in the page).
class	Assigns one or more class names to an element for styling or scripting.
style	Adds inline CSS styling directly to an element.
title	Provides extra information shown as a tooltip when hovering.
lang	Specifies the language of the element's content.
dir	Defines text direction (ltr for left-to-right, rtl for right-to-left).
data-*	Used to store custom data attributes for JavaScript or styling purposes.
href	Specifies the URL the link goes to.
target	Defines where to open the link (same tab, new tab, etc.).
rel	Describes the relationship between the current page and the linked resource.
cite	Indicates the source URL of a quotation or reference.
action	URL where the form data is sent after submission.
method	Defines how data is sent (GET or POST).
enctype	Specifies how form data is encoded when submitted
autocomplete	Enables or disables browser autofill for form fields.
novalidate	Disables built-in form validation.
name	Gives a name to the form or input for identification.
value	Sets the default or submitted value of an input.
for (label)	Links a label to a specific input element.
src	Specifies the image file source.
alt	Provides alternative text if the image cannot load.
usemap	Links an image to an image map.
shape	Defines the shape of a clickable area in an image map.
coords	Defines the coordinates for that shape.
src	Specifies the media file source.
controls	Shows playback controls (play, pause, volume).
autoplay	Starts media automatically when loaded
muted	Starts media with sound turned off.
height	Sets the height of the media display.
width	Sets the width of the media display.

colspan	Makes a cell span multiple columns.
rowspan	Makes a cell span multiple rows.
charset	Defines character encoding (like UTF-8).
http-equiv	Gives HTTP header information for the browser.
content	Provides the value for meta information.
media	Specifies the media/device the rule applies to
cx	X coordinate of a circle's center.
cy	Y coordinate of a circle's center.
r	Radius of a circle.
x	X position of an element.
y	Y position of an element.
rx	X-axis radius for rounded corners.
ry	Y-axis radius for rounded corners.
points	Defines points for shapes like polygons.
offset	Position of a gradient stop.
fill	Color used to fill a shape.
font-family	Defines font type in SVG text.

Chapter 02 : CSS

01*What is CSS?

CSS stands for Cascading Style Sheets. It is a language used to style and design web pages written in HTML.

CSS1 (1996)	Basic styling like colors, fonts, and simple layouts.
CSS2 (1998)	Added positioning, z-index, media types, and more layout control.
CSS2.1 (2011)	Fixed bugs and improved compatibility between browsers.
CSS3 (from 1999)	Introduced modern features

02*Types of CSS:

Inline CSS	Written directly inside an HTML element
Internal CSS	Written inside a <style> tag in the <head> section of the HTML file.
External CSS	Written in a separate .css file and linked to the HTML file

03*CSS syntax:

```
Selector {
  property1 : value1 ;  /* declaration */
  property2 : value2 ;  /* declaration */
  }
```

04*Types of Selectors:

A.Element Selector: Selects all <tag> elements
p {
  color: red;
}
 This is a paragraph
 Another paragraph
	B.ID Selector : Selects one element with a specific id
	#para1 {
	  color: blue;
	}
	 This is special

C.Class Selector : Selects all elements with a specific class
.center {
  text-align: center;
}
 Centered text
<h1 class="center">Title</h1>
	D.Element + Class Selector : Selects only <tag1> elements with class "classA"
	tag.classA {
	  color: green;
	}
	<tag1 class="classA">This works</tag1>
	<tag2 class="classA">This won't</tag2>

E.Multiple Classes : The element uses both classes
p.center {
  text-align: center;
}
p.large {
  font-size: 20px;
}
 Big centered text
	F.Universal Selector : Selects all elements
	* {
	  margin: 0;
	}

G.Group Selector : Applies the same style to multiple elements
h1, h2, p {
  color: purple;
}
 Title
 Subtitle
 Paragraph
	H.First Child : Selects a div if it is the first child inside its parent
	div:first-child {
	  background: yellow;
	}

I.Last Child : Selects a div if it is the last child inside its parent
div:last-child {
  background: orange;
}
	J.Descendant Selector : Selects all elements inside a
	div p {
	  color: brown;
	}
	 Inside div
	 Outside div

K.Media Query (Responsive Design) : Applies styles only when the screen width is 600px or less (mobile)
@media (max-width: 600px) {
  body {
  background: lightblue;
  }
}
	L.Pseudo-classes:
	Applies styles when the mouse pointer is over an element
	  button :hover {
	  background: red;
	  }

05*Text & Font Properties:

color	Sets the text color
text-align	Aligns text (left, center, right)
text-decoration	Adds decoration
text-decoration-line	Defines the type of line
text-decoration-style	Style of decoration
text-decoration-color	Color of decoration
text-transform	Changes text case
text-indent	Indents the first line
text-shadow	Adds shadow to text
letter-spacing	Controls space between letters
line-height	Controls space between lines
font-family	Sets the font type
font-size	Sets text size
font-style	Italic or normal text
font-weight	Boldness of text
font-variant	Small-caps text style

06*Box Model:

margin	Space outside element
margin-left/right/bottom	Specific outer spacing
padding	Space inside element
padding-top/right/bottom/ left	Specific inner spacing
width / height	Element size
min-width / max-width	Size limits
box-sizing	Controls how width/height are calculated

07*Border & Outline:

border	Sets border (width, style, color)
border-style	Solid, dotted, etc.
border-width	Thickness of border
border-color	Border color
border-radius	Rounded corners
border-bottom/border-left	Specific sides
outline	Outer line (outside border)
outline-style/width/ color	Outline settings
outline-offset	Space between border and outline

08*Background Properties:

background	Shorthand for all background properties
background-image	Adds image background
background-color	Background color
background-repeat	Repeat behavior
background-position	Image position
background-attachment	Fixed or scroll
background-origin	Starting area of background
background-clip	Where background is visible

09*Layout & Display:

display	Defines layout type
visibility	Show or hide element
opacity	Transparency level
overflow	Controls overflow content
float	Moves element left/right
clear	Controls float behavior

10*Positioning:

z-index	Controls stacking order
position	Defines positioning (static, relative…)
top / bottom / left / right	Position offsets

11*Flexbox & Grid:

order	Controls item order in flexbox
justify-content	Align items horizontally
flex-wrap	Wrap items to next line
grid-template-columns	Defines grid columns
grid-gap	Space between grid items
grid-area	Places item in grid

12*List Properties:

list-style	Shorthand for list styling
list-style-type	Type (disc, number…)
list-style-image	Custom image bullet
list-style-position	Bullet position

13*Table Properties:

border-collapse	Merge table borders
border-spacing	Space between cells
caption-side	Position of table caption
empty-cells	Show/hide empty cells

14*UI & Effects:

cursor	Mouse cursor style
transform	Rotate, scale, move element
animation	Adds animations

15*Counters & Variables:

counter-reset	Resets counter value
counter-increment	Increases counter
--primary-color	Custom variable
--primary-bg-color	Custom background variable
color	var(--primary-color);

16*Special / Meta:

inherit	Takes value from parent
initial-value	Default browser value
syntax	Defines how a property is written

Chapter 03 : JavaScript

01*About JavaScript:

JavaScript is a programming languages used to make websites interactive and dynamic
We use JavaScript to add interactivity and dynamic behavior to web pages , like buttons , animations , forms and changing content without reloading the page

02*Variables:

var:	Declaring a variable with function scope and allows re-declaring and reassignment
let :	allows reassignment but no re-declaring in the same scope
const :	Declares a block-scoped variable that cannot be reassigned after the initial value is set

03*DataTypes:

Number	Integers and decimals	let age= 25; let price = 19.23;
String	Text	let message = 'Hello World';
Boolean	true/false	let isLoggedIn = true;
Undefined	Declared but no value	let x;
Null	Intentional empty value	let data = null;
Symbol	Unique identifiers	let id = Symbol("user ID");
Bigint	Very large integers	let bigNumber = 9873827293;
NaN	Not a Number	let r = "abc"/2; console.log(r); //NaN
Object	The base non-primitive type	let person = { name: "jawad" , age:23 };
Array	Used to store multiple values in one variable	let numbers = [1,2,3,4]; let numbers = new Array(1,2,3,4);

Map	A collection of key-value	const user = new Map();
		user.set("name" , "ali");
		user.set(1 , "ID");

04*Strings:

length	Returns the number of characters in a string
charAt(i)	Returns the character at a specific index
charCodeAt(a)	Returns the unicode of the character at a specific index
at(a)	Returns the character at a specific index(supports negative index)
toUpperCase()	Converts the string to uppercase
toLowerCase()	Converts the string to lowercase
toLocaleUpperCase()	Converts the string to uppercase
indexOf("c")	Returns the position of the first occurrence of a value
lastIndexOf("c")	Returns the position of the last occurrence of a value
includes("word")	Checks if a string contains a value
startsWith("word")	Checks if a string starts with a specified value
endsWith("word")	Check if a string ends with a specified value
search(/a/)	Searches for a match using a regular expression and returns its position
match(/o/g)	Returns an array of matches using a regular expression
matchAll(/o/g)	Returns an iterator of all matches using a global regular expression
slice(a,b)	Extracts a part of a string and returns a new string
substring(a,b)	Extracts characters between two indices
substr(a,b)	Extracts a substring starting at a specifies index of a given length
replace("a","b")	Replaces the first occurrence of a specified value with another value
replaceAll("ab","cd")	Replaces all occurrences of a specified value with another value
split("symbol")	Splits a string into an array of substrings based on a separator
trim()	Removes whitespace from both ends of a string
trimStart()	Removes whitespace from the start of a string
trimEnd()	Removes whitespace from the end of a string
concat(" ","b")	Joins two or more strings and returns a new string
padStart(a,b)	Adds padding at the start of a string until it reaches a specified length
padEnd(a,"b")	Adds padding at the end of a string until it reaches a specified length
repeat(a)	Repeats to the string a specified number of times
localeCompare(b)	Compares two strings in a locale-sensitive manner
toString()	Converts a value to a string
valueOf()	Returns the primitive value of a string object

05*Operators in JavaScript:

Arithmetic	+ - * / % ++ --
Logical	&& || ^ |
Binary	& | ^ ~ << >>
Comparison	== != < > <= >= === !==
Assignment	= += -= /= &= |= ^= <<= >>=
String concatenation	+
Comments	// (single-line) /* */ (multi-line)
Others	. [] () {} ? : new

== (Loose Equality) : compares values after type conversion [console.log(5 == "5" && true == 1 && null == undefined) //true]

=== (Strict Equality) : compares both value and type without type conversion [console.log(5 === "5" || true ===1 || null === undefined) //false]

06*Conditional Statements:

if Statement:
if(condition) {
//code
}
	if-else Statement:
	if(condition) {
	//code1
	}
	else {
	//code2
	}

if/else if/else Statement:
if(condition1) {
//code1
}
else if(condition2) {
//code2
}
else {
//code3
}
	switch:
	switch(variable) {
	case result1 :
	//code1
	break;
	case result2 :
	//code2
	break;
	default:
	//code4
	break;
	}

Ternary Operator (?:) :
let result = (age>=18) ? "Adult" : "Minor" ;

False-like Conditions (False Values) :
False
0
-0
"" (empty string)
null
undefined
NaN

07*Loops:

while loop:
while(condition) {
//code
}
	do-while loop:
	do {
	//code
	} while(condition);

for loop:
for(initialization;condition;increment) {
//code
}
	forEach loop:
	array.forEach(function(element) {
	//code
	});

for-of loop:
for(const attribute of array) {
//code
}	

08*Functions:

function functionName(parameters) {
//code
return value;
}
	function functionName(parameters) {
	//code
	}

Global vs Function Scope:

```
let y = 5;
function example() {
   let x = 10;
   console.log(y); // accessible
 }
console.log(x); //error
```

09*Document Object Model(DOM):

Accessing HTML content:

document.getElementById()	Selects one single element by its unique id
document.getElementsByClassName()	Select multiple elements that share the same name
document.getElementsByTagName()	Selects all elements with the same HTML tag
document.querySelector(".class")	Selects the first matching
document.querySelectorAll(".class")	Selects all matching elements using a CSS selector

Manipulating HTML content:

.innerHTML	changes or reads HTML content inside an element (including tags)
.innerText	changes or reads only visible textt inside an element
.textContent	Changes or reads all text content , including hidden text
.src .href .alt .	changing attributes
attribute[n].style.color	changing style

10*Methods:

Timer/Event Methods:

setTimeout()	Executes a function once after a specified delay
clearTimeout()	Cancels a timeout created by setTimeout()
setInterval()	Repeats a function continously after a specified time interval
clearInterval()	Stops an interval created by setInterval()
appendChild()	Adds a new child element inside a parent element
removeChild()	Removes a child element form its parent element
setAttribute()	Sets or changes an attribute of an element
getAttribute()	Gets the value of an element's attribute
addEventListerner()	Attaches an event handler to an element
removeEventListener()	Removes an event handler from an element

Math Methods:

Math.random();	Returns a random number between 0 and 1
Math.floor()	Rounds a number down to the nearest integer
Math.ceil()	Rounds a number up to the nearest integer
Math.round()	Rounds a number to the nearest integer
Math.max()	Returns the largest number from given values
Math.min()	Returns the smallest number from given values

Array Methods:

push()	Adds one or more elements to the end of an array
pop()	Removes the last element from an array
shift()	Removes the first element form an array
unshift()	Adds one or more elements to the beginning of an array
map()	Creates a new array by applying a function to each element
filter()	Creates a new array with elements that match a condition
find()	Returns the first element that matches a condition
forEach()	Executes a function for each array element
includes()	Checks if an array contains a specific value
slice()	Returns a portion of an array without changing the original array
splice()	Adds, removes , or replaces elements in an array
sort()	Sorts the elements of an array

11*Browser Objects Model(BOM):

alert()	Displays a message box with an OK button
prompt()	Displays a dialog box that asks the user for input
confirm()	Displays a dialog box with OK and Cancel buttons
open()	Opens a new browser window or tab
close()	Closes the current browser
print()	Opens the browser print dialog to print the page

12*Objects:

An object in JavaScript is a data structure used to store related data as key-value pairs

```
const user = {
    name: “Ali” ,
    age: 20
  };
```

13*Inheritance:

Inheritance is a feature where a class can reuse properties and methods form another class

```
class Animal {
    eat() {
      console.log(“eating…”);
      }
    }
class Dog extends Animal {
    bark() {
       console.log(“barking…”);
      }
 }
const d = new Dog();
d.eat();
d.bark();
```

14*Maps:

A Map is a collection that stores key-values pairs where keys can be of any type

```
const map = new Map();
map.set(“name” , “Ali”);
map.set(1 , “number key”);
```

15*Arrow Functions:

Arrow functions are a shorter way to write functions using =>

```
const add = (a,b) => a+b ;
console.log(add(2,3));
```

Chapter 04 : ReactJS

01*About React:

React is a JavaScript library used for building fast and interactive user interfaces , espacially for single-page applications
We use it to build fast , reusable, and interactive user interfaces with efficient updates that improve performance in complex web applications

02*Default Content of App.jsx:

```
import { useState }  from 'react'
import reactLogo from './assests/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
const[count , setCount] = useState(0)
return (
   <>
    …
    <button Onclike() = { () => setCount( (count) => count+1) } > count is {count} </button>
    <p>Edit <code> src/App.jsx</code> and save to test HMR </p>
    …
    </>
  )
 }
 export default App;
```

useState : a React Hook used to create and manage state in a functional component

'react' : imports the main React library from node_modules
'./assets/react.svg' : imports a local SVG image file from the project folder

viteLogo : imports the Vite logo image to be used inside the component

'./App.css' : imports CSS styles for styling the App component

const [count, setCount] = useState(0) : creates a state variable count starting at 0 with a function to update it

onClick={() => setCount(count + 1)} : event handler that increases count by 1 every time the button is clicked

Notes:
- In React, component names (functions) must start with a capital letter (e.g., App, Header).

- HTML/CSS files and class names are case-sensitive, but CSS filenames are usually written in lowercase by convention (not capital).

- In React, JSX attributes use camelCase (e.g., onClick, not onclick).
useState returns an array: the first value is the state, the second is the function that updates it.

03*Replace the default content of the App.jsx file (src folder) :

To install the latest version, run the following from the terminal:
npm i react&latest react-dom&latest

main.jsx:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <App />
)
```

App.jsx:

```
function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  );
}
export default App;
```

index.html:

```
<div id="root"></div>
```

import React from 'react' : imports React library to use JSX and components

import ReactDOM from 'react-dom/client' : imports React tool that connects React to the browser DOM

ReactDOM.createRoot() : selects the HTML element where React will be mounted

.render() : renders the App component inside the root element

function App(): defines a React functional components called App

return() : returns the UI structure to be displayed

export default App : allows the App component to be imported in other files

04*Critical Path Vs Uncritical Path and Relative Path vs Absolute Path:

Critical Path : the essential files and code needed for the app to run (e.g, main.jsx , App.jsx); if they break , the app stops working

Non-critical Path : optional resources like images or CSS; they improve the app but don't stop it if they fail

Relative Path : a Path based on the current file location, using ./ or ../ (example: ./assets/react.svg)

Absolute Path : a path starting from the project root , using / (example: /vite.svg)

05*The container:

React uses a container to render HTML in a web page

index.html

```
<head>
<meta charset = “UTF-8” />
<link rel = “icon” type = “image/svg+xml” href = “/vite.svg” />
<title> Vite + React </title>
</head>
<body>
<div id=”root”></div>
<script type = “module” src = “/src/main.jsx” > </script>
```

Note : you should have a file called index.html

StrictMode:

A React Wrapper that activates additional checks and warning for its child components during development

<StrictMode> : A React wrapper that activates additional checks and warnings for its child components during development

main.jsx

```
import { StrictMode } from ‘react’
import { createRoot } from ‘react-dom/client’
import ‘./index.css’
import App from ‘./App.jsx’
createRoot(…).render(
   <StrictMode>
     <App />
   </StrictMode>
   )
```

The Root Node:

Can be called whatever you like

Display in the <header id = “sandy”> element

```
<body>
<header id= “sandy”> </header>
<script type = “module” src = “/src/main.jsx”> </script>
</body>
```

06*ES6(ECMAScript 6):

Classes:

A class is a type of function and the property’s are assigned inside a constructor method

```
class Car {
  constructor(name) {
    this.brand = name;
     }
  }
    present() {
      return ‘I have a ’+this.brand;
     }
const mycar = new Car(“Ford”);
mycar.present();
```

Export: (Spread Operator)

You can export a function or a variable from any variable - Destructing

export const name = "Tobias" **export const age = 18**	**const name = "Tobias"** **const age = 18** **export { name , age }**

Tagged Template Strings:

```
function highlight(strings, …fname) {
  let x = fname[0].toUpperCase();
  let y = fname[1].toUpperCase();
  return strings[0] + x + strings[1] + y + strings[2];
  }
let name 1 = "Name1" ;
let name2 = "Name2" ;
let text = highlight `Hello ${name1} and ${name2} , how are you?` ;
```

07*React JSX:

JSX allows us to write HTML elements in JS and place'em in the DOM without any createElement() and/ or appendChild() methods

```
import React from 'react' ;
import { createRoot , createElement } from 'react-dom/client'
const myElement = React.createElement('h1' , {} , 'I do not use JSX!');
createRoot(….).render(myElement) ;
const myElement = (
 <>
  <p>…</p>
  <p>…</p>
 </>
);
```

- Elements must be closed like <input type = "text" />
- Use Attribute className instead of class in JSX <h1 className = "myclass"> </h1>
- Comments in JSX are written with {/* */}

08*Object Properties:

```
function Car() {
 const myobj = { property1 : "value1" , property2: "value2" };
  return (
      <>
       <tag> … {myobj.property2} {myobj.property1} </tag>
      </>
    );
 }
```

09*React CSS:

Use the style attribute:

```
function Car() {
 const mystyles = {
  property1 : “value1” , property2 : “value2”};
 return (
   <>
      <tag style = {mystyles}>My car</tag>
   </>
   );
 }
```

10*Other Definitions:

changeColor()	will change the color property
setState()	update the component state and re-render the UI automatically
Destructing props	limit the properties a component receives by using destructuring

11*React Classes:

Components : Reusable building blocks in React (functions/classes)
state : object can contain as many properties
props : stands for properties

```
class Car extends React.Component {
 constructor() {
   super();
   this.state = { property : “value” };
 }
 render() {
   return <tag>…{ this.props.property }</tag>
     }
 }
createRoot(document.getElementById(‘root’)).render(
<Car />) ;
```

```
class Car extends React.Component {
 constructor (props) {
  super(props);
  }
 render() {
  return <tag>…{this.props.model}..</tag>
  }
}
```

Components in Components:

```
class classA extends React.Component {
 render() {
   return <tag>…</tag>
   }
 }
class classB extends React.Component {
 render() {
   return (
     <div>
       <tag>…</tag>
       <classA />
   </div>
     );
   }
}
createRoot(…).render(
<classB />
}
```

getDerivedStateFromProps() : is called right before the render method

```
class Header extends React.Component {
  constructor(props) {
    super(props);
    this.state = {favoritecolor : “value”};
    }
  static getDerivedStateFromProps(props , state) {
      return {favoritecolor : props.favcol };
  }
render() {
…
    }
}
```

componentDidMount() : called after the component is rendered

```
class Header extends React.Component {
    constructor(props) {
      super(props);
      this.state = {favoritecolor : “value” };
    }
    componentDidMount() {
      setTimeout(() => {
        this.setState( {favoritecolor : “value”} )
            } , 1000)
    }
render() {
    …
  }
}
```

shouldComponentUpdate() : can return a boolean value(with true default value)

```
class Header extends React.Component {
  constructor(props) {
    …
  }
  shouldComponentUpdate() {
    return false;
  }
  changeColor = () => {
      this.setState( {favoritecolor : “value”});
  }
render() {
…
    }
  }
```

getSnapshotBeforeUpdate() : access to the props and state before the update

```
class Header extends React.Component {
   constructor(props) {
     …
 }
 componentDidMount() {
    setTimeout( () => { this.setState({favoritecolor: “value”}) } , 1000)
  }
  getSnapshotBeforeUpdate(prevProps, prevState) {
     document.getElementById(“div1”).innerHTML = “…” + prevState.favoritecolor;
  }
  componentDidUpdate() {
     document.getElementById(“div2”).innerHTML = “…” + this.state.favoritecolor;
  }
  render() {
 }
}
createRoot(…).render(
<Header />);
```

…rest :

```
function Car( {color, brand , ..rest}) {
   return (
     <tag> {brand} {rest.model} {color}</tag>
   );
 }
createRoot(…).render(
  <Car brand = “Ford”  model = “Mustang”  color = “red”  year = {1969} />);
```

12*React Lists:

```
function MyCars() {
  const cars = [‘value1’ , ‘value2’ , ‘value3’ ];
    return(
      <>
        <tag>…</tag>
        <ul> { car.map( (car) => <li>I am a {car} </li>) } </ul>
      </>
      );
   }
createRoot(…).render(…) ;
```

Keys:

```
function MyCars() {
    const cars = [
        {id: ‘value’ , brand: ‘value’} , {id: 1002 , brand: ‘value’ , id:1003 , brand: ‘value’ } ];
    return (
      <>
        <tag>…</tag>
        <ul> {car.map( (car) => <li key={car.id} .. {car.brand} </li>) }</ul>
      </>
    );
}
createRoot(…).render(…) ;
```

13*React Forms:

```
function MyForm() {
  return (
    <form>
      <label> …
        <input type = “text” />
      </label>
    </form>
  )
}
createRoot(…).render(…);
```

```
import { useState }  from ‘react’ ;
import { createRoot }  from ‘react-dom/client’ ;
  function MyForm() {
      const [name , setName] = useState(“”) ;
  function handleChange(e) {
    setName(e.target.value) ;
  }
    return(
      <form>
        <label> …
          <input type = “text” value = {name} onChange = {handleChange} />
        </label> <p> Current value : {name} </p>
      </form>
  )
}
createRoot(…).render( … ) ;
```

Form Submit:

```
import { useState } from 'react' ;
import { createRoot } from 'react-dom/client' ;
function MyForm() {
   const [name , setName] = useState("");
function handleChange(e) {
   setName(e.target.value);
  }
function handleSubmit(e) {
   e.preventDefault() ;
   alert(name) ;
  }
return(
  <form onSumbit = {handel Submit}>
   <label>… <input type = "text" value = {name} onChange = {handleChange} /> </label>
   <input type = "submit" /> </form>
)
  }
  createRoot(…).render(…) ;
```

14*React Portal:

A Portal is a React method that is included in the react-dom package

```
import { createPortal } from 'react-dom';

function myChild() {

 return createPortal(

   <div>  Welcome  </div>,

   document.body

 );

}
```

Syntax:

```
import { createPortal } from 'react-dom';

createPortal(children, domNode)
```

Modal with Portal:

```
import { createRoot } from 'react-dom/client';
import { useState } from 'react';
import { createPortal } from 'react-dom';
function Modal({ isOpen, onClose, children }) {
 if (!isOpen) return null;
 return createPortal(
  <div style={{
   position: 'fixed',
   top: 0,  left: 0,  right: 0,  bottom: 0,  backgroundColor: 'rgba(0, 0, 0, 0.5)',  display: 'flex',
   alignItems: 'center',  justifyContent: 'center'  } } >
   <div style={{
    background: 'white',  padding: '20px',  borderRadius: '8px'  } } >
    {children}
    <button onClick={onClose}>Close</button>
   </div>
  </div>,
  document.body
 );}
function MyApp() {
 const [isOpen, setIsOpen] = useState(false);
 return (
  <div>
   <h1>My App</h1>
   <button onClick={() => setIsOpen(true)}>  Open Modal  </button>
   <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
    <h2>Modal Content</h2>
    <p>This content is rendered outside the App component!</p>
   </Modal>
  </div>
```

```
 );}
createRoot(document.getElementById('root')).render(
 <MyApp />)
```

15*React Suspense:

Suspense is a React feature that lets your components display an alternative HTML while waiting for code or data to load.

```
import { createRoot } from 'react-dom/client';import { Suspense } from 'react';import Fruits from './Fruits';
function App() {
 return (
  <div>
   <Suspense fallback={<div>Loading...</div>}>
    <Fruits />
   </Suspense>
  </div>
 );}
createRoot(document.getElementById('root')).render(
 <App />);
```

Lazy loading :

```
import { createRoot } from 'react-dom/client';import { Suspense, lazy } from 'react';
const Cars = lazy(() => import('./Cars'));
function App() {
 return (
  <div>
   <Suspense fallback={<div>Loading...</div>}>
    <Cars />
   </Suspense>
  </div>
 );}
```

16*React Styling:

Inline Styling:

```
const Header = ()  => {
  return (
    <>
      <tag style = { {property: “value” }}>Hello Style!</tag>
<tag>…<tag>
</>
); }
```

JavaScript Object:

```
const Header() = () => {
  const myStyle = {
      color : “value1” , backgroundColor : “value2” , padding : “value3” , fontFamilly : “value4” } ;
return (
    <>
      <tag style = {myStyle}>…</tag>
    </>
  );
}
```

CSS Stylesheet:

MyStylesheet.css	index.html

body {
background-color : “value” ;
color : “value” ;
padding : value;
font-family : value;
text-align : value;
}
	<!Doctype html>
	<html lang = “en”>
	<body>
	
	<script type = “module” src = “/src/main.jsx”></script>
	</body> </html>

main.jsx
import { createRoot } from ‘react-dom/client’ ; import ‘./MyStylesheet.css’ ; const Header = () => { return (<> <tag>…</tag> <tag>…</tag> </>); } createRoot(…).render (<Header />) ;

17*React Modules:

`.module.css` extension and can be used by importing it into your React file(s).

main.jsx

```
import { createRoot } from 'react-dom/client' ;
import styles from './Button.module.css' ;
function App() {
return(
<div>
<button className = {styles.mybutton}> My Button </button>
</div>
); }
createRoot(…).render(…) ;
```

Button.module.css

```
.mybutton {
  padding : value;
  border: value;
  border-radius : value;
  cursor : value;
}
```

index.html

```
<!Doctype html>
  <html lang = "en">
    <body>
      <div id = "root"> </div>
      <script type = "module" src ="/src/main.jsx"></script>
    </body>
  </html>
```

React CSS in Js:

To install styled-components , run the following command:

```
npm install styled-components
```

```
import {createRoot } from 'react-dom/client' ;
import styled from 'styled-components' ;
const MyHeader = styled.h1 `padding: value ;  background-color: value;  color: value; ` ;
 function App() {
   return (
     <>
       <MyHeader>…</MyHeader>
     </>
     );
  }
 createRoot(…).render() ;
```

18*React Forward Ref:

forwardRef lets your component pass a reference to one of its children. It's like giving a direct reference to a DOM element inside your component.

```
import { createRoot } from 'react-dom/client' ;
import { forwardRef , useRef } from 'react' ;
  const MyInput = forwardRef( (props , ref) => ( <input ref = {ref} {...props} />));
  function App() {
     const inputRef = useRef() ;
     const focusInput = () => { inputRef.current.focus(); } ;
     return (
     <div>
       <MyInput ref = {inputRef} placeholder = "Type here.." />
       <button onClick = {focusInput} > Focus Input </button>
     </div>
  );
}
createRoot(...).render(...) ;
```

19*React Sass:

Sass is a CSS pre-processor , files are executed on the server and sends CSS to the browser
Insall the package

```
npm install sass
```

main.jsx

```
Import { createRoot } from 'react-dom/client' ;
Import './MyStyle.scss' ;
Function MyHeader() {
Return (
<h1>My Header</h1>
); }
createRoot(...).render(...) ;
```

MyStyle.scss

```
$myColor : red;
h1 {
Color : $myColor;
}
```

index.html

```
<!Doctype html>
<html lang = "en">
<body>
<div id = "root"></div>
<script type= "module" src = "/src/main.jsx"></script>
</body>
</html>
```

20*React Router:

BrowserRounter: enables navigation and routing in a React application using the browser

Routes: A container that groups all Route components together

Route: Defines which component should render for a specific URL path

Link: creates navigation links without reloading the page

Install React Router:

```
npm install react-router-dom
```

```
import { createRoot } from 'react-dom/client' ;
import { BrowserRounter , Routes , Route , Link } from 'react-router-dom' ;
 function Home() {
    return <h1>Home Page</h1>; }
 function About() {
  return <h1>About Page</h1>; }
 function Contact() {
  return <h1>Contact Page</h1>; }
 function App() {
  return (
   <BrowserRouter>
    <nav>
     <Link to = "/">Home</Link> | {"  "}
     <Link to="/about">About</Link> | {"  "}
     <Link to = "/contact">Contact</Link>
    </nav>
    <Routes>
       <Route path= "/" element = {<Home />} />
       <Route path= "/about" element = {<About /> } />
       <Route path = "/contact" element = { <Contact />} />
    </Routes>
   </BrowserRouter>
   );
 }
createRoot(…).render(..) ;
<App />
);
```

21*React Transitions:

The useTransition hook helps you keep your React app responsive during heavy updates.

```
import { createRoot } from 'react-dom/client' ;
import { useState , useTransition } from 'react' ;
function SearchResults ( {query} ) {
  const items = [] ;
  if(query) {
     for(let I = 0 ; I< 1000 ; I++) {
     items.push(<li key = {I}>Result for {query} - {I} </li>);
       }
   }
   return <ul>{items}</ul>;
  }
function App() {
   const [input , setInput ] = useState(' ');
   const [query , setQuery] = useState(' ');
   const [isPending , startTransition] = useTransition();
   const handleChange = (e) => {
    setInput(e.target.value);
    startTransition( ( ) => {
         setQuery(e.target.value);
      });
   };
   return (
      <div>
        <input type = "text"  value = {input} onChange = {handleChange} placeholder = "Type to search…" />
{isPending && <p>Loading results…</p>}
<SearchResults query = {query} />
</div>
); }
createRoot(…).render(
<App />);
```

Chapter 05: Bootstrap

01*What is Bootstrap?

Bootstrap is a free front-end framework for faster and easier web developmentBootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables …. Bootstrap also gives you the ability to easily create responsive designs

02*Bootstrap5 Latest Compiled:

Latest Compiled and minified CSS:

<link href = "https://cdn.jsdeliv.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel = "stylesheet">

Latest Compiled JavaScript:

<script src = "https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>

First Bootstrap Page: (with Container Class)

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
 </head>
 <body>
  <div class="container-fluid">
  <h1>…</h1>
  <p>…</p>
  </div>
 </body>
</html>
```

03*Grid Basic:

```
<!DOCTYPE html>
 <html lang="en">
 <head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
 </head>
```

```
 <body>
  <div class="container-fluid mt-3">
  <h1>Three equal width columns</h1>
  <p>Note: Try to add a new div with class="col" inside the row class - this will create four equal-width columns.</p>
  <div class="row">
   <div class="col p-3 bg-primary text-white">.col</div>
   <div class="col p-3 bg-dark text-white">.col</div>
   <div class="col p-3 bg-primary text-white">.col</div>
   </div>
  </div>
 </body>
</html>
```

```
<body>
 <div class="container-fluid mt-3">
 <h1>Two Unequal Responsive Columns</h1>
 <p>Resize the browser window to see the effect.</p>
 <p>The columns will automatically stack on top of each other when the screen is less than 576px wide.</p>
 <div class="row">
  <div class="col-sm-4 p-3 bg-primary text-white">.col</div>
  <div class="col-sm-8 p-3 bg-dark text-white">.col</div>
  </div>
 </div>
</body>
```

04*Typography:

Style the Heading element:

```
<body>
 <div class="container mt-3">
   <h1>…</h1>
    <p>…</p>
 <h1 class="display-1">Display 1</h1>
 <h1 class="display-2">Display 2</h1>
 <h1 class="display-3">Display 3</h1>
 <h1 class="display-4">Display 4</h1>
 <h1 class="display-5">Display 5</h1>
 <h1 class="display-6">Display 6</h1>
 </div>
</body>
```

Style the <abbr> element:

```
<body>
<div class="container mt-3">
 <h1>…</h1>
 <p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
</div>
</body>
```

Style the <blockquote> Element:

```
<blockquote class="blockquote">
   <p>…</p>
   <footer class="blockquote-footer">From WWF's website</footer>
 </blockquote>
</div>
```

Style the Description List:

```
<div class="container mt-3">
 <h1>Description Lists</h1>
 <p>The dl element indicates a description list:</p>
 <dl>
   <dt>Coffee</dt>
   <dd>- black hot drink</dd>
   <dt>Milk</dt>
   <dd>- white cold drink</dd>
 </dl>
</div>
```

Style the Code element:

```
<div class="container mt-3">
 <h1>Code Snippets</h1>
 <p>The following HTML elements: <code>span</code>, <code>section</code>, and <code>div</code>
defines a section in a document<p>
</div>
```

Style the Kbd element:

```
<div class="container mt-3">
  <h1>Keyboard Inputs</h1>
  <p>Use <kbd>ctrl + p</kbd> to open the Print dialog box.</p>
</div>
```

Style the pre element:

```
<div class="container mt-3">
<h1>Multiple Code Lines</h1>
<pre>
Text in a pre element
is displayed in a fixed-width
font, and it preserves
both      spaces and
line breaks.
</pre>
```

05*Colors:

Text Color:

```
<div class="container mt-3">
  <h2>Contextual Colors</h2>
  <p>…</p>
  <p class="text-muted">...</p>
  <p class="text-primary">...</p>
  <p class="text-success">...</p>
  <p class="text-info">...</p>
  <p class="text-warning">...</p>
  <p class="text-danger">...</p>
  <p class="text-secondary">...</p>
  <p class="text-dark">...</p>
  <p class="text-body">...</p>
  <p class="text-light">...</p>
  <p class="text-white">...</p>
</div>
```

Background Text Color:

```
<div class="container mt-3">
  <h2>Opacity Text Colors</h2>
  <p>…</p>
  <p class="text-black-50">Black text with 50% opacity</p>
  <p class="text-white-50 bg-dark">White text with 50% opacity</p>
</div>
```

Contextual Background:

```
<div class="container mt-3">
  <h2>Contextual Backgrounds</h2>
  <p>...</p>
  <div class="bg-primary p-3"></div>
  <div class="bg-success p-3"></div>
  <div class="bg-info p-3"></div>
  <div class="bg-warning p-3">…</div>
  <div class="bg-danger p-3></div>
  <div class="bg-secondary p-3”></div>
  <div class="bg-dark p-3"></div>
  <div class="bg-light p-3"></div>
</div>
```

06*Bootstarp Tables:

Better Style:

```
<div class="container mt-3">
  <h2>…</h2>
  <table class="table table-striped">
    <thead>
      <tr>
        <th>Firstname</th>
        <th>Lastname</th>
        <th>Email</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>John</td>
        <td>Doe</td>
        <td>john@example.com</td>
      </tr>
      <tr>
        <td>Mary</td>
        <td>Moe</td>
        <td>mary@example.com</td>
      </tr>
      <tr>
        <td>July</td>
        <td>Dooley</td>
        <td>july@example.com</td>
      </tr>
    </tbody>
  </table>
</div>
```

Add Border:

```
<div class="container mt-3">
  <h2>…</h2>
  <table class="table table-bordered">
    <thead>
      <tr>
        <th>Firstname</th>
        <th>Lastname</th>
        <th>Email</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>John</td>
        <td>Doe</td>
        <td>john@example.com</td>
      </tr>
      <tr>
        <td>Mary</td>
        <td>Moe</td>
        <td>mary@example.com</td>
      </tr>
      <tr>
        <td>July</td>
        <td>Dooley</td>
        <td>july@example.com</td>
      </tr>
    </tbody>
  </table>
</div>
```

Hover Rows	Dark Table
  Hover Rows … <table class="table table-hover"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>John</td> <td>Doe</td> <td>john@example.com</td> </tr> <tr> <td>July</td> <td>Dooley</td> <td>july@example.com</td> </tr> </tbody> </table> 	 Black/Dark Table … <table class="table table-dark"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>John</td> <td>Doe</td> <td>john@example.com</td> </tr> <tr> <td>July</td> <td>Dooley</td> <td>july@example.com</td> </tr> </tbody> </table>

Dark Striped Table	Hoverable Dark Table
  Dark Striped Table Combine .table-dark and .table-striped to create a dark, striped table: <table class="table table-dark table-striped"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>John</td> <td>Doe</td> <td>john@example.com</td> </tr> <tr> <td>July</td> <td>Dooley</td> <td>july@example.com</td> </tr> </tbody> </table> 	 Hoverable Dark Table The .table-hover class adds a hover effect (grey background color) on table rows: <table class="table table-dark table-hover"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>Mary</td> <td>Moe</td> <td>mary@example.com</td> </tr> <tr> <td>July</td> <td>Dooley</td> <td>july@example.com</td> </tr> </tbody> </table>

Borderless Table	Contextual Classes Table
  Borderless Table … <table class="table table-borderless"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>John</td> <td>Doe</td> <td>john@example.com</td> </tr> <tr> <td>Mary</td> <td>Moe</td> <td>mary@example.com</td> </tr> <tr> <td>July</td> <td>Dooley</td> <td>july@example.com</td> </tr> </tbody> </table> 	 Contextual Classes … <table class="table"> <thead> <tr> <th>Firstname</th> <th>Lastname</th> <th>Email</th> </tr> </thead> <tbody> <tr> <td>Default</td> <td>Defaultson</td> <td>def@somemail.com</td> </tr> <tr class="table-primary"> <td>Primary</td> <td>Joe</td> <td>joe@example.com</td> </tr> <tr class="table-success"> <td>Success</td> <td>Doe</td> <td>john@example.com</td> </tr> </tbody> </table>

07*Images:

The .rounded-circle class shapes the image to a circle:

```
<div class="container mt-3">
 <h2>Circle</h2>
 <p>The .rounded-circle class shapes the image to a circle:</p>
 <img src="cinqueterre.jpg" class="rounded-circle" alt="Cinque Terre" width="304" height="236">
</div>
```

The .img-thumbnail class shapes the image to a thumbnail (bordered):

```
<div class="container mt-3">
 <h2>Thumbnail</h2>
 <p>The .img-thumbnail class creates a thumbnail of the image:</p>
 <img src="cinqueterre.jpg" class="img-thumbnail" alt="Cinque Terre" width="304" height="236">
</div>
```

Float an image to the left with the .float-start class or to the right with .float-end or center:

```
<div class="container mt-3">
 <h2>Aligning images</h2>
 <p>Use the float classes to float the image to the left or to the right:</p>
 <img src="paris.jpg" class="float-start" alt="Paris" width="304" height="236">
 <img src="paris.jpg" class="float-end" alt="Paris" width="304" height="236">
 <img src="paris.jpg" class="mx-auto d-block">
</div>
```

08*Animated Alerts:

Closing Alerts:

```
<div class="container mt-3">
  <h2>Alerts</h2>
  <p>The button with class="btn-close" and data-bs-dismiss="alert" is used to close the alert box.</p>
  <p>The alert-dismissible class aligns the button to the right.</p>
  <div class="alert alert-success alert-dismissible">
    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    <strong>Success!</strong> This alert box could indicate a successful or positive action.
  </div>
  <div class="alert alert-info alert-dismissible">
    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    <strong>Info!</strong> This alert box could indicate a neutral informative change or action.
  </div>
 </div>
</div>
```

09*Button:

Button Classes:

```
<div class="container mt-3">
  <h2>Button Styles</h2>
  <button type="button" class="btn">Basic</button>
  <button type="button" class="btn btn-primary">Primary</button>
  <button type="button" class="btn btn-secondary">Secondary</button>
  <button type="button" class="btn btn-success">Success</button>
  <button type="button" class="btn btn-info">Info</button>
  <button type="button" class="btn btn-warning">Warning</button>
  <button type="button" class="btn btn-danger">Danger</button>
  <button type="button" class="btn btn-dark">Dark</button>
  <button type="button" class="btn btn-light">Light</button>
  <button type="button" class="btn btn-link">Link</button>
</div>
```

```
<a href="#" class="btn btn-success">Link Button</a>
<button type="button" class="btn btn-success">Button</button>
<input type="button" class="btn btn-success" value="Input Button">
<input type="submit" class="btn btn-success" value="Submit Button">
<input type="reset" class="btn btn-success" value="Reset Button">
```

Button Outline:

```
<button type="button" class="btn btn-outline-primary">Primary</button>
<button type="button" class="btn btn-outline-secondary">Secondary</button>
<button type="button" class="btn btn-outline-success">Success</button>
<button type="button" class="btn btn-outline-info">Info</button>
<button type="button" class="btn btn-outline-warning">Warning</button>
<button type="button" class="btn btn-outline-danger">Danger</button>
<button type="button" class="btn btn-outline-dark">Dark</button>
<button type="button" class="btn btn-outline-light text-dark">Light</button>
```

Button Sizes:

```
<div class="container mt-3">
  <h2>Button Sizes</h2>
  <button type="button" class="btn btn-primary btn-lg">Large</button>
  <button type="button" class="btn btn-primary btn-md">Default</button>
  <button type="button" class="btn btn-primary btn-sm">Small</button>
</div>
```

Block Level Button:

```
<div class="d-grid">
  <button type="button" class="btn btn-primary">Full-Width Button</button>
</div>
```

Active/Disabled Button:

```
<div class="container mt-3">
  <h2>Button States</h2>
  <button type="button" class="btn btn-primary">Primary Button</button>
  <button type="button" class="btn btn-primary active">Active Primary</button>
  <button type="button" class="btn btn-primary" disabled>Disabled Primary</button>
  <a href="#" class="btn btn-primary disabled">Disabled Link</a>
</div>
```

Spinner Button:

```
<div class="container mt-3">
  <h2>Spinner Buttons</h2>
  <p>Add spinners to buttons:</p>
  <button class="btn btn-primary">
    <span class="spinner-border spinner-border-sm"></span>
  </button>
  <button class="btn btn-primary">
    <span class="spinner-border spinner-border-sm"></span>
    Loading..
  </button>
  <button class="btn btn-primary" disabled>
    <span class="spinner-border spinner-border-sm"></span>
    Loading..
  </button>
  <button class="btn btn-primary" disabled>
    <span class="spinner-grow spinner-grow-sm"></span>
    Loading..
  </button>
</div>
```

Button Groups Horizontal :

```
<div class="btn-group">
  <button type="button" class="btn btn-primary">Apple</button>
  <button type="button" class="btn btn-primary">Samsung</button>
  <button type="button" class="btn btn-primary">Sony</button>
</div>
```

Button Groups Vertical :

```
<div class="btn-group-vertical">
  <button type="button" class="btn btn-primary">Apple</button>
  <button type="button" class="btn btn-primary">Samsung</button>
  <button type="button" class="btn btn-primary">Sony</button>
</div>
```

Nesting Button Groups & Dropdown Menus:

```
<div class="btn-group">
  <button type="button" class="btn btn-primary">Apple</button>
  <button type="button" class="btn btn-primary">Samsung</button>
  <div class="btn-group">
     <button type="button" class="btn btn-primary dropdown-toggle" data-bs
toggle="dropdown">Sony</button>
    <div class="dropdown-menu">
      <a class="dropdown-item" href="#">Tablet</a>
      <a class="dropdown-item" href="#">Smartphone</a>
    </div>
  </div>
</div>
```

10*Badges:

Badges (New) :

```
<div class="container mt-3">
  <h2>Badges</h2>
  <h1>Example heading <span class="badge bg-secondary">New</span></h1>
  <h2>Example heading <span class="badge bg-secondary">New</span></h2>
</div>
```

Contextual Badges:

```
<div class="container mt-3">
  <h2>Contextual Badges</h2>
  <span class="badge bg-primary">Primary</span>
  <span class="badge bg-secondary">Secondary</span>
  <span class="badge bg-success">Success</span>
  <span class="badge bg-danger">Danger</span>
  <span class="badge bg-warning">Warning</span>
  <span class="badge bg-info">Info</span>
  <span class="badge bg-light">Light</span>
  <span class="badge bg-dark">Dark</span>
</div>
```

11*Bars:

Basic Process Bar:

```
<div class="container mt-3">
  <h2>Basic Progress Bar</h2>
  <p>…</p>
  <div class="progress">
    <div class="progress-bar" style="width:70%"></div>
  </div>
</div>
```

Process Bar Label:

```
<div class="container mt-3">
  <h2>Progress Bar Height</h2>
  <p>…</p>
  <div class="progress" style="height:10px">
    <div class="progress-bar" style="width:40%;"></div>
  </div>
  <br>
  <div class="progress" style="height:20px">
    <div class="progress-bar" style="width:50%;"></div>
  </div>
  <br>
  <div class="progress" style="height:30px">
    <div class="progress-bar" style="width:60%;"></div>
  </div>
</div>
```

```
<div class="progress">
  <div class="progress-bar" style="width:70%">70%</div>
</div>
```

Colored Progress Bar:

```
<!-- Blue -->
<div class="progress">
  <div class="progress-bar" style="width:10%"></div>
</div>
<!-- Green -->
<div class="progress">
  <div class="progress-bar bg-success" style="width:20%"></div>
</div>
<!-- Turquoise -->
<div class="progress">
  <div class="progress-bar bg-info" style="width:30%"></div>
</div>
<!-- Orange -->
<div class="progress">
  <div class="progress-bar bg-warning" style="width:40%"></div>
</div>
<!-- Red -->
<div class="progress">
  <div class="progress-bar bg-danger" style="width:50%"></div>
</div>
<!-- White -->
<div class="progress border">
  <div class="progress-bar bg-white" style="width:60%"></div>
</div>

<!-- Grey -->
<div class="progress">
  <div class="progress-bar bg-secondary" style="width:70%"></div>
</div>
```

Striped Progress Bar:

```
<div class="progress">
  <div class="progress-bar progress-bar-striped" style="width:40%"></div>
</div>
```

Animated Progress Bar:

```
 <div class="progress">
   <div class="progress-bar progress-bar-striped progress-bar-animated" style="width:40%"></div>
 </div>
```

12*Spinners:

Loader:

```
<div class="spinner-border"></div>
```

Colored Spinners:

```
<div class="spinner-border text-muted"></div>
<div class="spinner-border text-primary"></div>
<div class="spinner-border text-success"></div>
<div class="spinner-border text-info"></div>
<div class="spinner-border text-warning"></div>
<div class="spinner-border text-danger"></div>
<div class="spinner-border text-secondary"></div>
<div class="spinner-border text-dark"></div>
<div class="spinner-border text-light"></div>
```

Growing Spinners:

```
<div class="spinner-grow text-muted"></div>
<div class="spinner-grow text-primary"></div>
<div class="spinner-grow text-success"></div>
<div class="spinner-grow text-info"></div>
<div class="spinner-grow text-warning"></div>
<div class="spinner-grow text-danger"></div>
<div class="spinner-grow text-secondary"></div>
<div class="spinner-grow text-dark"></div>
<div class="spinner-grow text-light"></div>
```

13*Pagination:

Basic Pagination: (horizontal)

```
<ul class="pagination">
   <li class="page-item"><a class="page-link" href="#">Previous</a></li>
   <li class="page-item"><a class="page-link" href="#">1</a></li>
   <li class="page-item"><a class="page-link" href="#">2</a></li>
   <li class="page-item"><a class="page-link" href="#">3</a></li>
   <li class="page-item"><a class="page-link" href="#">Next</a></li>
  </ul>
</div>
```

Active State:

```
<ul class="pagination">
  <li class="page-item"><a class="page-link" href="#">Previous</a></li>
  <li class="page-item"><a class="page-link" href="#">1</a></li>
  <li class="page-item active"><a class="page-link" href="#">2</a></li>
  <li class="page-item"><a class="page-link" href="#">3</a></li>
  <li class="page-item"><a class="page-link" href="#">Next</a></li>
</ul>
```

14*List Group:

To create a basic list group, use an <ul> element with class .list-group, and <li> elements with class .list-group-item:

```
<div class="container mt-3">
  <h2>Basic List Group</h2>
  <ul class="list-group">
    <li class="list-group-item">First item</li>
    <li class="list-group-item">Second item</li>
    <li class="list-group-item">Third item</li>
  </ul>
</div>
```

Use the .active class to highlight the current item:

```
<ul class="list-group">
<li class="list-group-item active">Active item</li>
<li class="list-group-item">Second item</li>
<li class="list-group-item">Third item</li>
</ul>
```

The .disabled class adds a lighter text color to the disabled item. And when used on links, it will remove the hover effect:

```
<div class="list-group">
  <a href="#" class="list-group-item disabled">Disabled item</a>
  <a href="#" class="list-group-item disabled">Disabled item</a>
  <a href="#" class="list-group-item">Third item</a>
</div>
```

Use the .list-group-flush class to remove some borders and rounded corners:

```
<ul class="list-group list-group-flush">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
  <li class="list-group-item">Fourth item</li>
</ul>
```

Use the .list-group-numbered class to create list items with numbers in front of them:

```
<ol class="list-group list-group-numbered">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
</ol>
```

if you want the list items to display horizontally instead of vertically (side by side instead of on top of each other), add the .list-group-horizontal class to .list-group:

```
<ul class="list-group list-group-horizontal">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
  <li class="list-group-item">Third item</li>
  <li class="list-group-item">Fourth item</li>
</ul>
```

Combine .badge classes with utility/helper classes to add badges inside the list group:

```
<ul class="list-group">
  <li class="list-group-item d-flex justify-content-between align-items-center">  Inbox
    <span class="badge bg-primary rounded-pill">12</span>
  </li>
  <li class="list-group-item d-flex justify-content-between align-items-center"> Ads
    <span class="badge bg-primary rounded-pill">50</span>
  </li>
  <li class="list-group-item d-flex justify-content-between align-items-center"> Junk
    <span class="badge bg-primary rounded-pill">99</span>
  </li>
</ul>
```

15*Bootstrap Cards:

Basic Card:

```
<div class="container mt-3">
  <h2>Basic Card</h2>
  <div class="card">
    <div class="card-body">Basic card</div>
  </div>
</div>
```

Titles, text, and links:

```
<div class="container mt-3">
  <h2>Card titles, text, and links</h2>
  <p>…</p>
  <div class="card">
    <div class="card-body">
      <h4 class="card-title">Card title</h4>
      <p class="card-text">Some example text. Some example text.</p>
      <a href="#" class="card-link">Card link</a>
      <a href="#" class="card-link">Another link</a>
    </div>
  </div>
</div>
```

Add .card-img-top or .card-img-bottom to an <img> to place the image at the top or at the bottom inside the card. Note that we have added the image outside of the .card-body to span the entire width:

```
<div class="card" style="width:400px">
  <img class="card-img-top" src="img_avatar1.png" alt="Card image">
  <div class="card-body">
    <h4 class="card-title">John Doe</h4>
    <p class="card-text">Some example text.</p>
    <a href="#" class="btn btn-primary">See Profile</a>
  </div>
</div>
```

Card Image Overlays:

```
<div class="container mt-3">
  <h2>Card Image Overlay</h2>
  <p>Turn an image into a card background and use .card-img-overlay to overlay the card's text:</p>
  <div class="card img-fluid" style="width:500px">
    <img class="card-img-top" src="../bootstrap4/img_avatar1.png" alt="Card image" style="width:100%">
    <div class="card-img-overlay">
      <h4 class="card-title">John Doe</h4>
      <p class="card-text">...</p>
      <a href="#" class="btn btn-primary">See Profile</a>
    </div>
  </div>
</div>
```

16*Bootstrap Dropdown:

Basic Dropdown:

```
<div class="dropdown">
 <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown">
  Dropdown button
 </button>
 <ul class="dropdown-menu">
  <li><a class="dropdown-item" href="#">Link 1</a></li>
  <li><a class="dropdown-item" href="#">Link 2</a></li>
  <li><a class="dropdown-item" href="#">Link 3</a></li>
 </ul>
</div>
```

Drop Down Divider:

```
<li><hr class="dropdown-divider"></hr></li>
```

Dropdown Header:

```
<h2>Dropdowns</h2>
 <p>The .dropdown-header class is used to add headers inside the dropdown menu:</p>
 <div class="dropdown">
  <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown">
   Dropdown button
  </button>
  <ul class="dropdown-menu">
   <li><h5 class="dropdown-header">Dropdown header 1</h5></li>
   <li><a class="dropdown-item" href="#">Link 1</a></li>
   <li><a class="dropdown-item" href="#">Link 2</a></li>
   <li><a class="dropdown-item" href="#">Link 3</a></li>
   <li><h5 class="dropdown-header">Dropdown header 2</h5></li>
   <li><a class="dropdown-item" href="#">Another link</a></li>
  </ul>
 </div>
</div>
```

Disable and Active items:

```
 <div class="dropdown">
  <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown">
   Dropdown button
  </button>
  <ul class="dropdown-menu">
   <li><a class="dropdown-item" href="#">Normal</a></li>
   <li><a class="dropdown-item active" href="#">Active</a></li>
   <li><a class="dropdown-item disabled" href="#">Disabled</a></li>
  </ul>
 </div>
```

Dropdown Position:

Dropdown left	**Dropdown right**
 <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown"> Dropend </button> <ul class="dropdown-menu"> Normal Active Disabled 	 <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown"> Dropstart </button> <ul class="dropdown-menu"> Normal Active Disabled

Dropup:

```
<div class="dropup">
  <button type="button" class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown">
    Dropup button
  </button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Link 1</a></li>
    <li><a class="dropdown-item" href="#">Link 2</a></li>
    <li><a class="dropdown-item" href="#">Link 3</a></li>
  </ul>
 </div>
```

Grouped Buttons with a Dropdown:

```
<div class="btn-group">
 <button type="button" class="btn btn-primary">Apple</button>
 <button type="button" class="btn btn-primary">Samsung</button>
 <div class="btn-group">
        <button type="button" class="btn btn-primary dropdown-toggle" data-bs
toggle="dropdown">Sony</button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Tablet</a></li>
    <li><a class="dropdown-item" href="#">Smartphone</a></li>
  </ul>
 </div>
</div>
```

17*Bootstrap Collapsible:

```
<div class="container mt-3">
  <h2>Simple Collapsible</h2>
  <p>Click on the button to toggle between showing and hiding content.</p>
  <button type="button" class="btn btn-primary" data-bs-toggle="collapse" data-bs-target="#demo">Simple collapsible</button>
  <div id="demo" class="collapse">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
  </div>
</div>
```

You can add the .show class to show the content by default:

```
<div class="container mt-3">
  <h2>Show Collapsible Content By Default</h2>
  <p>Add the show class next to the collapse class to show the content by default.</p>
  <p>Click on the button to toggle between showing and hiding content.</p>
  <button type="button" class="btn btn-primary" data-bs-toggle="collapse" data-bs-target="#demo">Simple collapsible</button>
  <div id="demo" class="collapse show">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
  </div>
</div>
```

18*Bootstrap Navs:

Tabs:

```
<ul class="nav nav-tabs">
  <li class="nav-item">
    <a class="nav-link active" href="#">Active</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled" href="#">Disabled</a>
  </li>
</ul>
```

Pills:

```
<ul class="nav nav-pills">
  <li class="nav-item">
    <a class="nav-link active" href="#">Active</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled" href="#">Disabled</a>
  </li>
</ul>
```

Toggleable / Dynamic Tabs:

```
<div class="container mt-3">
  <h2>Toggleable Tabs</h2>
  <br>
  <ul class="nav nav-tabs" role="tablist">
    <li class="nav-item">
      <a class="nav-link active" data-bs-toggle="tab" href="#home">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" data-bs-toggle="tab" href="#menu1">Menu 1</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" data-bs-toggle="tab" href="#menu2">Menu 2</a>
    </li>
  </ul>
  <div class="tab-content">
    <div id="home" class="container tab-pane active"><br>
      <h3>HOME</h3>
      <p>…</p>
    </div>
    <div id="menu1" class="container tab-pane fade"><br>
      <h3>Menu 1</h3>
      <p>...</p>
    </div>
    <div id="menu2" class="container tab-pane fade"><br>
      <h3>Menu 2</h3>
      <p>…</p>
    </div>
  </div>
</div>
```

Toggleable / Dynamic Pills:

```
<div class="container mt-3">
 <h2>Toggleable Pills</h2>
 <br>
 <ul class="nav nav-pills" role="tablist">
  <li class="nav-item">
   <a class="nav-link active" data-bs-toggle="pill" href="#home">Home</a>
  </li>
  <li class="nav-item">
   <a class="nav-link" data-bs-toggle="pill" href="#menu1">Menu 1</a>
  </li>
  <li class="nav-item">
   <a class="nav-link" data-bs-toggle="pill" href="#menu2">Menu 2</a>
  </li>
 </ul>
 <div class="tab-content">
  <div id="home" class="container tab-pane active"><br>
   <h3>HOME</h3>
   <p>...</p>
    </div>
    <div id="menu1" class="container tab-pane fade"><br>
        <h3>Menu 1</h3>
        <p>…</p>
       </div>
       <div id="menu2" class="container tab-pane fade"><br>
        <h3>Menu 2</h3>
        <p>...</p>
       </div>
     </div>
    </div>
```

19*Bootstrap Navbars:

Navbar Forms and Buttons:

```
<nav class="navbar navbar-expand-sm navbar-dark bg-dark">
 <div class="container-fluid">
  <a class="navbar-brand" href="javascript:void(0)">Logo</a>
  <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#mynavbar">
   <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="mynavbar">
   <ul class="navbar-nav me-auto">
    <li class="nav-item">
     <a class="nav-link" href="javascript:void(0)">Link</a>
    </li>
    <li class="nav-item">
     <a class="nav-link" href="javascript:void(0)">Link</a>
    </li>
    <li class="nav-item">
     <a class="nav-link" href="javascript:void(0)">Link</a>
    </li>
   </ul>
   <form class="d-flex">
    <input class="form-control me-2" type="text" placeholder="Search">
    <button class="btn btn-primary" type="button">Search</button>
   </form>
  </div>
 </div>
</nav>
```

```
<div class="container-fluid mt-3">
  <h3>Navbar Forms</h3>
  <p>You can also include forms inside the navigation bar.</p>
</div>
```

20*Bootstrap Cursor:

```
<div id="demo" class="carousel slide" data-bs-ride="carousel">
  <!-- Indicators/dots -->
  <div class="carousel-indicators">
    <button type="button" data-bs-target="#demo" data-bs-slide-to="0" class="active"></button>
    <button type="button" data-bs-target="#demo" data-bs-slide-to="1"></button>
    <button type="button" data-bs-target="#demo" data-bs-slide-to="2"></button>
  </div>
  <!-- The slideshow/carousel -->
  <div class="carousel-inner">
    <div class="carousel-item active">
      <img src="la.jpg" alt="Los Angeles" class="d-block" style="width:100%">
    </div>
    <div class="carousel-item">
      <img src="chicago.jpg" alt="Chicago" class="d-block" style="width:100%">
    </div>
    <div class="carousel-item">
      <img src="ny.jpg" alt="New York" class="d-block" style="width:100%">
    </div>
  </div>
  <!-- Left and right controls/icons -->
  <button class="carousel-control-prev" type="button" data-bs-target="#demo" data-bs-slide="prev">
    <span class="carousel-control-prev-icon"></span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-target="#demo" data-bs-slide="next">
    <span class="carousel-control-next-icon"></span>
  </button>
</div>
<div class="container-fluid mt-3">
  <h3>Carousel Example</h3>
  <p>The following example shows how to create a basic carousel with indicators and controls.</p>
</div>
```

21*Bootstrap Modals:

Create a Modal:

```
<div class="container mt-3">
  <h3>Modal Example</h3>
  <p>Click on the button to open the modal.</p>
  <button type="button" class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#myModal">
    Open modal
  </button>
</div>
<div class="modal" id="myModal">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h4 class="modal-title">Modal Heading</h4>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        Modal body..
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-danger" data-bs-dismiss="modal">Close</button>
      </div>
    </div>
  </div>
</div>
```

Fullscreen Modals:

```
<div class="modal-dialog modal-fullscreen">
```

22*Bootstrap Toasts:

```
<div class="container mt-3">
  <h3>Toast Example</h3>
  <p>...</p>
  <p>...</p>
  <div class="toast show">
    <div class="toast-header">
      <strong class="me-auto">Toast Header</strong>
      <button type="button" class="btn-close" data-bs-dismiss="toast"></button>
    </div>
    <div class="toast-body">
      <p>…</p>
    </div>
  </div>
</div>
```

23*Bootstrap OffCanvas:

OffCanvas:

```
<div class="offcanvas offcanvas-start" id="demo">
  <div class="offcanvas-header">
    <h1 class="offcanvas-title">Heading</h1>
    <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
  </div>
  <div class="offcanvas-body">
    <p>Some text lorem ipsum.</p>
    <p>Some text lorem ipsum.</p>
    <p>Some text lorem ipsum.</p>
    <button class="btn btn-secondary" type="button">A Button</button>
  </div>
</div>
<div class="container-fluid mt-3">
  <h3>Offcanvas Sidebar</h3>
  <p>Offcanvas is similar to modals, except that it is often used as a sidebar.</p>
  <button class="btn btn-primary" type="button" data-bs-toggle="offcanvas" data-bs-target="#demo">
    Open Offcanvas Sidebar
  </button>
</div>
```

Bottom OffCanvas:

```
<div class="offcanvas offcanvas-bottom" id="demo">
  <div class="offcanvas-header">
    <h1 class="offcanvas-title">Heading</h1>
    <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
  </div>
  <div class="offcanvas-body">
    <p>Some text lorem ipsum.</p>
    <button class="btn btn-secondary" type="button">A Button</button>
  </div>
</div>

<div class="container-fluid mt-3">
  <h3>Bottom Offcanvas</h3>
  <p>The .offcanvas-bottom class positions the offcanvas at the bottom of the page.</p>
  <button class="btn btn-primary" type="button" data-bs-toggle="offcanvas" data-bs-target="#demo">
    Toggle Bottom Offcanvas
  </button>
</div>
```

Dark OffCanvas:

```
<div class="offcanvas offcanvas-end" id="demo">
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="offcanvas"></button>
```

24*Bootstrap Forms:

Forms:

```
<div class="container mt-3">
  <h2>Stacked form</h2>
  <form action="/action_page.php">
    <div class="mb-3 mt-3">
      <label for="email">Email:</label>
      <input type="email" class="form-control" id="email" placeholder="Enter email" name="email">
    </div>
    <div class="mb-3">
      <label for="pwd">Password:</label>
      <input type="password" class="form-control" id="pwd" placeholder="Enter password" name="pswd">
    </div>
    <div class="form-check mb-3">
      <label class="form-check-label">
        <input class="form-check-input" type="checkbox" name="remember"> Remember me
      </label>
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>
```

Floating Labels:

```
<div class="container mt-3">
  <h2>Floating Labels - Inputs</h2>
  <p>Click inside the input field to see the floating label effect:</p>
  <form action="/action_page.php">
    <div class="form-floating mb-3 mt-3">
      <input type="text" class="form-control" id="email" placeholder="Enter email" name="email">
      <label for="email">Email</label>
    </div>
    <div class="form-floating mt-3 mb-3">
      <input type="text" class="form-control" id="pwd" placeholder="Enter password" name="pswd">
      <label for="pwd">Password</label>
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>
```

Select Menus:

```
<div class="container mt-3">
  <h2>Floating Labels - Select</h2>
  <p>You can also use "floating-labels" on select menus. However, they will not float/get animated. The label
will always appear in the top left corner, inside the select menu:</p>
  <form action="/action_page.php">
    <div class="form-floating mb-3 mt-3">
      <select class="form-select" id="sel1" name="sellist">
        <option>1</option>
        <option>2</option>
        <option>3</option>
        <option>4</option>
      </select>
      <label for="sel1" class="form-label">Select list (select one):</label>
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>
```

25*Form Validation:

```
<div class="container mt-3">
  <h3>Form Validation</h3>
  <p>Try to submit the form.</p>
  <form action="/action_page.php" class="was-validated">
    <div class="mb-3 mt-3">
      <label for="uname" class="form-label">Username:</label>
      <input type="text" class="form-control" id="uname" placeholder="Enter username" name="uname" required>
      <div class="valid-feedback">Valid.</div>
      <div class="invalid-feedback">Please fill out this field.</div>
    </div>
    <div class="mb-3">
      <label for="pwd" class="form-label">Password:</label>
      <input type="password" class="form-control" id="pwd" placeholder="Enter password" name="pswd" required>
      <div class="valid-feedback">Valid.</div>
      <div class="invalid-feedback">Please fill out this field.</div>
    </div>
    <div class="form-check mb-3">
      <input class="form-check-input" type="checkbox" id="myCheck"  name="remember" required>
      <label class="form-check-label" for="myCheck">I agree on blabla.</label>
      <div class="valid-feedback">Valid.</div>
      <div class="invalid-feedback">Check this checkbox to continue.</div>
    </div>
  <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>
```

Chapter 06 : Git & GitHub

01*What is Version Control?

Definition:

Version control is a system that records changes tO ties over time so that you can recall specific versions later.
It's likea time machine for your code.

Why it's important:

Track changes and history of a project
Revert to previous versions if something breaks
Collaborate with multiple developers without conflicts

Example (conceptual):

Imagine you have a file index. html:
Version 1: <h1>Hello World</h1>
Version 2: <h1>Hello Web Voyage</h1>
Version 3: <hl>Welcome to Web Development</h1>
A version control systemn lets you go back to any previous version whenever needed.

02*What is Git?

Definition:

Git is a popular distributed version control system. It allows multiple developers to work on the same project
simultaneously while keeping a full history of changes.

Key points:

-Local and distributed: each developer has a full copy of the project history
-Fast and reliable
-Works with branches to experiment safely without affecting main code

Example commands: (Bash)

```
git init # Initialize a Git repository in a folder
git status # Check the status of your files
git add index.html # Stage a file for committing
git commit -m "Update header" # Save changes with a message
```

Conceptual example:
you are writing code in a project folder. Git records each checkpoint you commit so you can always go back or share it with others.

03*What is GitHub?

Definition:
GitHub is a cloud-based platform that hosts Git repositories online. It allows developers to collaborate. Share and manage projects.

Why GitHub is useful:
-Backup your projects online
-Collaborate with other developers easily
-Track issues, pull requests, and contributions

Example workflow:
-Create a repository on GitHub
-Push your local Git commits to GitHub:

```
git remote add origin https://github.com/username/my-project.git
git push -u origin main
```

Collaborators can clone, make changes, and push updates

Conceptual example:
GitHub is like a library of your projects where you can safely store them, show your work, and work with a team anywhere in the world.

04*Installing Git:

Definition:
Git needs to be installed on your computer before you can use it. It works on Windows, macOS, and Linux.
How to install:
Windows: Download from git-scm.com and run the installer.
macOS: Install via Homebrew.
Linux: Install via package manage.
Verify installation:

```
git --version
```

This should show the installed Git version.

05*Basic Git Workflow:

Concept:
Git workflow is the sequence of steps you use to track and manage changes in your proiect
Typical workflow steps:
Initialize a repository (if not already):

```
git init
```

Check status of files:

```
git status
```

Stage files for commit:

```
git add filename.txt
git add. # add all files
```

Commit changes with a message:

```
git commit -m "Initial commit"
```

Push changes to remote repository (GitHub):

```
git push -u origin main
```

Example scenario:

You create index.html and style.css. You add them to Git, commit, and push to GitHub. Later, if you make changes, you repeat the workflow to save new versions.

06*Creating a Repository:

Definition:

A repository (repo) is a storage space for your project, either locally or on GitHub.

Creating a local repository:

```
mkdir my-project
cd my-project
git init
```

This creates a .git folder that tracks all changes in the project.

Creatinga GitHub repository:

-Go to GitHub: New Repository
-Give it a name (e.g., web-voyage)
-Choose Public or Private
-Click Create Repository

Connecting local repo to GitHub:

```
git remote add origin
git branch -M main
git push -u origin main
```

Conceptual example:

Local repo: your personal working folder
GitHub repo: online backup & collaboration space

07*Tracking Changes:

Definition:

Tracking changes in Git means monitoring which files have been modified, added, or deleted si commit.

Example scenario:

You have index.html. You add a new paragraph:

```
<p>Welcome to Web Voyagel</p>
```

```
git status
# Shows: modified: index.html
git diff
# Shows the exact line added
```

Concept:
Tracking lets you review changes before saving them permanently with a commit.

08*Branches:

Definition:
Branches allow you to develop features or experiment in isolation without affecting the main code (usually main branch).

Why use branches:
-Safe testing
-Parallel development
-Organized workflow

Commands:

```
git branch feature-navbar # create a branch
git checkout feat ure-navbar # switch to branch
git branch # list branches
```

Example scenario:
main branch: stable website
feature-navbar branch: trying a new navigation bar
Changes in feature-navbar won't affect main until merged

09*Merging Branches:

Definition:
Merging means combining changes from one branch into another, usually from a feature branch into main.

Commands:

```
git checkout main   # switch to main branch
git merge feature-navbar # merge changes from feature-navbar
```

Example scenario:
You finished the navigation bar in feature-navbar. After testing, you merge it into main. Now main has the new navbar.

Handling conflicts:
If two branches changed the same line, Git will ask you to resolve the conflict manually before completing the merge.

10*Remote Repositories:

Definition:
A remote repository is a version of your project hosted online, typically on platforms like GitHub Git or Bitbucket. It allows you to collaborate with others and backup your code.

Common commands:

```
git remote -v # Show remote repositories
git remote add origin https://github.com/username/web-voyage.git
git push -u origin main
git pull origin main
```

Example scenario:
You worked locally on index.html
Pushed your changes to GitHub with git push
Another developer can pull your changes with git pull

11*GitHub Workflow:

Definition:
GitHub workflow is a standard process for collaborating on projects using Git and GitHub.

Steps:
Clone repository:

```
git clone https://github.com/username/web-voyage.git
```

Create a new branch:

```
git checkout -b feature-header
```

Make changes locally:
Commit changes:

```
git add.
git commit -m "Add new header section"
```

Push branch to GitHub:

```
git push origin feature-header
```

Create Pull Request (PR) on GitHub for review
Merge PR into main branch after approval

Conceptual examnple:
Multiple developers can work simultaneously on different branches and merge safely without overwriting each other's work.

12*gitignore:

Definition:
A .gitignore file tells Git which files or folders to ignore and not track in version control.
Why it's important:
Avoid committing sensitive information (e.g., passwords, APNkeys)
Skip unnecessary files (e.g., node modules, build folders)

Example gitignore file:

```
node modules/
.env
dist/
*log
```

Example scenario:
You have node_modules folder and .env file for environment variables
By adding them to .gitignore, Git won't track them, keeping the repo clean and secure

13*Commnon Git Commands:

git init	Initialize a local repository
git clone <url>	Copy a remote repository locally
git status	Check changes and status
git add <file>	Stage files for commit
git commit -m "message"	Save changes
git push	Send changes to remote
git pull	Fetch & merge from remote
git branch <name>	Create a branch
git checkout <branch>	Switch branch
git merge <branch>	Merge branch into current branch

14*Why Git is Essential for Developers:

-Tracks changes and allows reverting to previous versions
-Enables safe collaboration with multiple developers
-Provides remote backup
-Organizes features, fixes, and experiments using branches

Example:

Restore a deleted file:

```
git checkout HEAD~1 index.html
```